Cognitive Tokens: A Runtime Token Layer for Tokenomics-Based AI Systems

Series: Tokenomics Journal Series

Status: Research/runtime draft; public-safe theory surface with sealed-core boundaries

Primary contribution: This paper extends Tokenomics from cognitive return measurement into an explicit cognitive-token layer for AI systems. The central move is to distinguish native model tokens from higher-order runtime tokens that can be minted, scored, routed, stored, hashed, and reused across agents.

Abstract

Large language models already consume tokens, but the token units exposed by model providers are mostly accounting units for input, output, cost, and context length. They do not directly represent decision value, action usefulness, risk reduction, memory formation, provenance, or system-level control. Tokenomics proposes that AI systems should not optimize for brevity alone, but for cognitive return per token: the measurable usefulness created by each unit of expression and computation. This paper introduces the next implementation layer: cognitive tokens.

A cognitive token is a typed runtime object created from an AI interaction, artifact, workflow, or reasoning product. Unlike native model tokens, cognitive tokens are explicit and inspectable. They can represent text spans, semantic units, formulas, decisions, actions, risks, memories, provenance, release boundaries, compression signals, governance constraints, system workflows, agent capability bundles, and proof receipts. This allows Tokenomics to move from a scoring theory into a ledgered runtime substrate. A system can mint cognitive tokens from a session, score them using a token value function, route them to specialized controllers, store them in a hash-chained ledger, extract reusable memory, and optionally generate proof receipts for downstream notary or PARALLAX-style infrastructure.

The paper defines the token taxonomy, formal model, lifecycle, formulas, runtime architecture, vault structure, and boundary rules needed to implement the cognitive-token layer. It also establishes a critical safety and precision boundary: these tokens are not the opaque internal tokens used by model providers, and proof tokens are not financial instruments unless a separate governed and counsel-reviewed issuance layer is created.

1. Problem

Current AI token accounting answers a narrow question: how many provider-level tokens were consumed? That is useful for cost and context control, but insufficient for agentic systems. A thousand output tokens can be valuable if they solve a high-risk decision, produce a reusable workflow, preserve provenance, and reduce failure modes. Ten output tokens can be waste if they hide uncertainty or force the user to ask again.

The Tokenomics measurement layer already reframes the question. Instead of asking only how many tokens were used, it asks what cognitive return was created per token. The missing layer is objecthood. If token value is real, the system needs explicit objects that carry that value through runtime. Those objects are cognitive tokens.

2. Core distinction

Tokenomics requires three token planes.

2.1 Native model tokens

Native model tokens are the internal units consumed by a model API or model runtime. They determine cost, context length, truncation, and throughput. They are usually not semantically stable enough to serve as knowledge or governance objects by themselves.

2.2 Cognitive runtime tokens

Cognitive runtime tokens are explicit objects created by a Tokenomics engine. They represent the value-bearing units produced or consumed by an AI system. They are typed, scored, hashable, auditable, and routeable.

2.3 Proof and settlement tokens

Proof tokens are hashable receipt objects derived from cognitive-token bundles or ledgers. They can support provenance, notary preparation, repository publication, or future ledger anchoring. They are not automatically tradeable assets, securities, exchange instruments, or legal notarizations.

3. Cognitive token definition

A cognitive token is defined as:

$$
ct_i = \langle id_i, type_i, plane_i, payload_i, source_i, score_i, metadata_i, parents_i, hash_i \rangle
$$

Where:

- id_i is a stable token identifier.
- $type_i$ is the cognitive token class.
- $plane_i$ is native, runtime, or proof/settlement.
- $payload_i$ is the content or object represented by the token.
- $source_i$ identifies the source session, artifact, paper, repo, or workflow.
- $score_i$ stores value and risk measurements.
- $metadata_i$ stores routing, boundary, and operational tags.
- $parents_i$ stores upstream token or artifact hashes.
- $hash_i$ commits the canonical token object.

4. Token classes

The first production taxonomy contains fifteen token classes.

The taxonomy is intentionally not limited to text. A working AI system uses math, memory, risk, action, provenance, and governance tokens constantly. The goal is to make them explicit.

5. Token value formula

Each cognitive token receives a value score:

$$
TV(ct_i)=DQ_i+ACT_i+RISK_i+REUSE_i+LEARN_i+CMP_i-WASTE_i
$$

With optional confidence adjustment:

$$
TV_c(ct_i)=TV(ct_i) \cdot Conf_i
$$

Where:

- DQ is decision quality contribution.
- ACT is action usefulness.
- $RISK$ is risk reduction.
- $REUSE$ is future reuse value.
- $LEARN$ is learning or future behavior improvement.
- CMP is compression or meaning-preservation contribution.
- $WASTE$ is redundancy, filler, false precision, or distraction.
- $Conf$ is confidence in the score.

6. Cognitive return and CRPT

For a set of cognitive tokens C:

$$
CR(C)=\sum_{ct_i \in C} TV_c(ct_i)
$$

Cognitive return per native token is:

$$
CRPT=\frac{CR(C)}{T_{prompt}+T_{output}}
$$

This connects provider-level token counts to the cognitive-token layer. Native tokens still matter, but they become the denominator of a value system rather than the whole accounting model.

7. Salience and budget control

Before generation, a Tokenomics system should estimate the salience of each topic, task, or candidate reasoning branch:

$$
S_i=\alpha U_i+\beta R_i+\gamma M_i+\delta T_i+\epsilon N_i-\zeta K_i
$$

Where urgency, risk, mission relevance, time sensitivity, and novelty increase salience, while already-known context lowers salience.

Budget allocation follows:

$$
B_i=B_{total}\cdot\frac{S_i}{\sum_j S_j}
$$

This means the system does not merely compress after writing. It governs token creation before and during reasoning.

8. Token lifecycle

The cognitive-token lifecycle has nine stages.

1. Observe native model token counts, cost, latency, and context pressure.
2. Segment the interaction into candidate spans, claims, formulas, decisions, actions, risks, and memories.
3. Classify each unit into a token class.
4. Score each token using the value function.
5. Route the token to the proper runtime controller.
6. Store selected tokens in the ledger or vault.
7. Compress and consolidate reusable tokens.
8. Generate proof receipts for controlled release or priority evidence.
9. Feed high-value memory/governance tokens into future AI behavior.

9. Runtime architecture

The proposed implementation has the following modules.

9.1 Token Observer

Reads prompt/output token counts, latency, cost, model identity, and session metadata.

9.2 Token Minter

Converts content into cognitive tokens. It creates token objects, classifies them, attaches source references, and computes hashes.

9.3 Token Scorer

Computes token value, cognitive return, CRPT, and waste penalties.

9.4 Token Governor

Sets budgets before generation and enforces salience-weighted allocation.

9.5 Token Router

Routes decision tokens to decision systems, action tokens to workflow systems, risk tokens to red-team guards, memory tokens to memory stores, and proof tokens to ledgers.

9.6 Ledger Vault

Stores a hash-chained JSONL ledger of token events. Each record commits the current token hash and previous record hash.

9.7 Agent Injector

Packages selected memory, system, governance, and agent tokens into an ingestion prompt or callable artifact that another AI can read and adopt.

9.8 PARALLAX Proof Bridge

Optionally turns hash bundles into proof receipt tokens. This is a proof and audit surface only unless separate governance and counsel approve asset issuance.

10. Whole-system tokens

The strongest form is the Agent Token. An Agent Token packages a reusable behavior policy, not just a text span. For example:

$$
AGT = \langle role, triggers, tools, constraints, memory\_rules, scoring\_rules, release\_boundary, verification \rangle
$$

This is how a system can inject a whole operating behavior into another AI. It is not magic. It works because the receiving AI is given a compact, structured, high-salience object that tells it what to do, when to do it, how to score itself, and what boundaries not to cross.

11. Vault and ledger design

A ledger record is:

$$
lr_i=H(index_i,timestamp_i,event_i,token_i,tokenHash_i,previousRecordHash_i)
$$

The ledger head is the final record hash. A proof receipt token can commit:

$$
PFR=H(bundleHash,ledgerHead,timestamp,issuer,releaseLabel)
$$

The vault keeps sealed and public tokens separated. Public papers may expose class definitions and formulas. The sealed vault keeps raw provenance, private memory, source artifacts, unreleased benchmarks, and agent-injection material.

12. Public vs sealed boundary

Public-safe material:

- token taxonomy,
- formulas,
- sample schemas,
- dummy ledgers,
- high-level benchmark method,
- proof receipt concept.

Sealed material:

• raw internal source artifacts,
• private memory tokens,
• private agent prompts,
• unpublished provenance ledgers,
• unreleased benchmark datasets,
• keys, signatures, credentials, or operational bridge configurations,
• financial-token issuance or clearing designs before counsel review.

13. Relation to PARALLAX

PARALLAX-style infrastructure can become useful at the proof, audit, and settlement-adjacent layer. Tokenomics creates the cognitive-token bundle. The ledger vault hashes it. A proof receipt token commits it. A future PARALLAX bridge can anchor that receipt.

This should be kept separate from financial issuance. A proof receipt token says: this cognitive-token bundle existed in this form at this time under this release label. It does not say that the token is tradeable, valuable as an investment, cleared, regulated, or legally notarized.

14. Implementation pattern

A minimal runtime implementation can be written as follows.

1. Accept a session JSON containing text spans, formulas, decisions, actions, risks, memories, boundaries, systems, and proof receipts.
2. Mint cognitive tokens from each unit.
3. Score each token.
4. Append each token to a hash-chained ledger.
5. Verify ledger integrity.
6. Emit a bundle hash.
7. Render selected tokens into public paper, local vault pages, or an agent ingestion prompt.
8. Keep sealed material out of the public surface.

15. Evaluation

The cognitive-token layer should be evaluated using these metrics:

$$
TokenomicGain = \frac{Score_B}{Tokens_B} - \frac{Score_A}{Tokens_A}
$$

$$
LedgerIntegrity = \begin{cases} 1 & \text{if all record hashes verify} \\ 0 & \text{otherwise} \end{cases}
$$

$$
ReuseYield = \frac{MEM+SYS+AGT+GOV}{TotalCognitiveTokens}
$$

$$
RiskPreservation = \frac{RSK_{retained}}{RSK_{identified}}
$$

$$
ProofReadiness = BoundaryScore \cdot ProvenanceCompleteness \cdot HashCompleteness
$$

16. Research implications

The main research implication is that AI systems need a second token layer above native model tokens. Native tokens measure computation and context. Cognitive tokens measure function, value, and control. Once the higher layer exists, a system can ask better questions:

• Which tokens changed the decision?
• Which tokens created reusable memory?

• Which tokens prevented a failure?
• Which tokens should be preserved across sessions?
• Which tokens should become public research?
• Which tokens must remain sealed?
• Which bundles are ready for hashing or notary preparation?

17. Conclusion

Tokenomics begins as a theory of cognitive return per token. The cognitive-token layer turns that theory into runtime machinery. It creates explicit objects from the valuable parts of AI work, scores them, routes them, stores them, and prepares them for reuse and proof. This is the step that lets Tokenomics operate both as a research-facing framework and as an injectable substrate for AI systems.

The correct claim is not that we create the model provider's internal tokens. The correct claim is stronger and cleaner: we create a higher-order token layer that governs what native tokens are used for, what value they generate, what should be remembered, what should be released, and what should be sealed.

Appendix A - Cognitive Token Classes

ID	Class	Public/Sealed	Use
TXT	Text Token	PUBLIC	surface text span accounting; links native model token counts to semantic/value layers
SEM	Semantic Token	PUBLIC	compressed meaning unit; used for memory and retrieval efficiency
MAT	Math Token	PUBLIC	formula, variable, parameter, equation, scoring model, benchmark expression
DEC	Decision Token	PUBLIC	decision-changing claim or recommendation; high value when it changes action
ACT	Action Token	PUBLIC	callable or executable next move; feeds workflow engine
RSK	Risk Token	PUBLIC	failure mode, uncertainty, boundary, guardrail, red-team signal
MEM	Memory Token	SEALED	reusable rule, template, pattern, skill, doctrine fragment, future behavior update
PRV	Provenance Token	SEALED	origin, source, timestamp, hash, authorship, proof signal
BND	Boundary Token	SEALED	sealed/public/control-release label; prevents overrelease
CMP	Compression Token	PUBLIC	meaning-preservation and compression-audit signal
GOV	Governance Token	SEALED	budget, route, policy, permissions, constraint, release rule
SYS	System Token	PUBLIC	workflow, module interface, callable artifact, runtime component
AGT	Agent Token	SEALED	agent role/capability bundle; can inject whole-system behavior into other AIs
VAL	Value Token	PUBLIC	derived score object representing cognitive return, CRPT, or token gain
PFR	Proof Receipt Token	SEALED	hash/notary/ledger receipt; optional PARALLAX bridge object, not an issued security